

PROGRAMMING LANGUAGES

ADVANCED GRAMMAR DESIGN IN ANTLR

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026



© Michele Pasqua, 2026

ADVANCED ANTLR USAGE



RECURRING PROGRAMMING LANGUAGE PATTERNS

Sequence of tokens or subphrases

- ▶ Assignment in Java: `x = 3 + 5;`

Reasonable ANTLR rule

```
assign  :  ID '=' exp ';' ;
```

RECURRING PROGRAMMING LANGUAGE PATTERNS (CONT'D)

Sequence of tokens or subphrases with terminator

- ▶ Sequence of commands in Java: `x = 3 + 5; obj.aMethod();`

Reasonable ANTLR rule

```
seq  : statement ';' seq |      ;  
seq  : (statement ';' )*      ; // advanced pattern  
seq  : (statement ';' )+      ; // non-empty
```

Where `[]*` matches `[]` zero, one, or more times

While `[]+` matches `[]` one or more times

RECURRING PROGRAMMING LANGUAGE PATTERNS (CONT'D)

Sequence of tokens or subphrases with separator

- ▶ Method arguments in Java: 3, x, new Object()

Reasonable ANTLR rules

```
args  :  exp tail                ;  
tail  :  ',' args |              ;  
args  :  exp (',' exp)*          ;    // advanced pattern  
args  :  (exp (',' exp)*)?       ;    // potentially empty
```

Where `[]?` matches `[]` or skip it

RECURRING PROGRAMMING LANGUAGE PATTERNS (CONT'D)

Token dependency

- ▶ Array access delimiters in Java: `arr[4]`

Reasonable ANTLR rule

```
access  :  ID '[' exp ']' ;
```

RECURRING PROGRAMMING LANGUAGE PATTERNS (CONT'D)

Nested phrases

- ▶ Nested class in Java: `class A { class B {} }`

Reasonable ANTLR rule

```
class : 'class' ID '{' (class | method | field) '}' ;
```

Where `(□ | □)` is a subrule matching multiple alternatives

PRECEDENCE

Real world programming languages handle **precedence**

$3 + 4 * 5 + ++x \leq 10 + 5 \% 2$

should be interpreted as

$((3 + (4 * 5)) + (++x)) \leq (10 + (5 \% 2))$

Without the need for explicit parenthesization

PRECEDENCE IN ANTLR

ANTLR resolves ambiguity in favor of the alternative given first

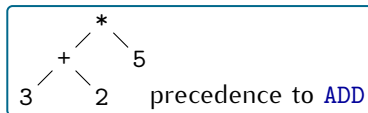
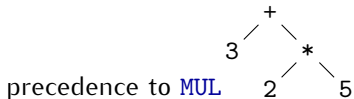
```
exp : NAT | exp ADD exp | exp MUL exp ;
```

```
ADD : '+' ;
```

```
MUL : '*' ;
```

```
NAT : '0' | [1-9] [0-9]* ;
```

How to parse 3+2*5 ?



PRECEDENCE IN ANTLR

ANTLR resolves ambiguity in favor of the alternative given first

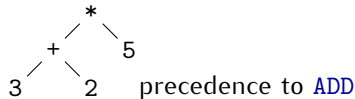
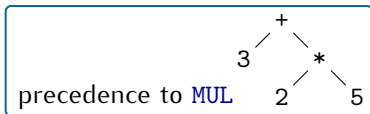
```
exp : NAT | exp MUL exp | exp ADD exp ;
```

```
ADD : '+' ;
```

```
MUL : '*' ;
```

```
NAT : '0' | [1-9] [0-9]* ;
```

How to parse 3+2*5 ?



PRECEDENCE IN ANTLR (CONT'D)

Some operators may have the same precedence

```
exp : exp ADD exp | exp SUB exp  
    | NAT  
    ;  
ADD : '+' ;  
SUB : '-' ;  
NAT : '0' | [1-9] [0-9]* ;
```

PRECEDENCE IN ANTLR (CONT'D)

Some operators may have the same precedence

```
exp : exp op=(ADD | SUB) exp      # addSub
    | NAT                          # nat
    ;

ADD : '+' ;
SUB : '-' ;
NAT : '0' | [1-9][0-9]* ;
```

```
@Override
public Integer visitAddSub(Parser.AddSubContext ctx) {
    switch (ctx.op.getType()) {
        case Parser.ADD :
            return visit(ctx.exp(0)) + visit(ctx.exp(1));
        case Parser.SUB :
            return visit(ctx.exp(0)) - visit(ctx.exp(1));
    }
    return 0; // unreachable code
}
```

ASSOCIATIVITY

Real world programming languages handle **associativity**

2^3^4 should be interpreted as $2^{(3^4)}$

and

$10 \% 4 \% 2$ should be interpreted as $(10 \% 4) \% 2$

Since exponentiation (^) is right-associative while modulo (%) is left-associative

- Addition is both, i.e., $1+3+5$ can be interpreted either as $1+(3+5)$ or $(1+3)+5$

ASSOCIATIVITY IN ANTLR

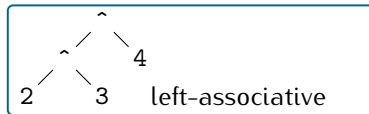
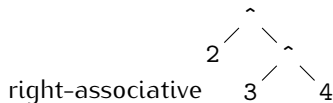
By default, ANTLR associates to the left unless we use the **assoc** option

```
exp : NAT | exp POW exp ;
```

```
POW : '^' ;
```

```
NAT : '0' | [1-9][0-9]* ;
```

How to parse 2^3^4 ?



ASSOCIATIVITY IN ANTLR

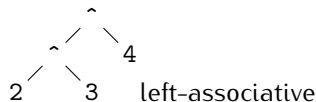
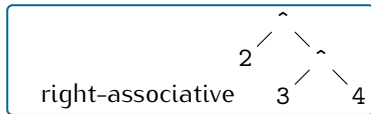
By default, ANTLR associates to the left unless we use the **assoc** option

```
exp : NAT | <assoc=right> exp POW exp ;
```

```
POW : '^' ;
```

```
NAT : '0' | [1-9][0-9]* ;
```

How to parse 2^3^4 ?



COMMON LEXICAL STRUCTURES

Identifiers (variables, function names, etc.)

- Match one or more uppercase and lowercase letters

ID : [a-zA-Z]+ ;

Natural numbers (0, 1, 2, etc.)

- Match 0 or any sequence of digits starting with 1

NAT : '0' | [1-9] [0-9]* ;

GRAMMAR FRAGMENTS

Integer numbers (0, 1, 2, -3, etc.)

- ▶ Match 0 or any sequence of digits starting with 1, potentially prefixed by -

```
INT           : NAT | '-' POS ;
fragment NAT  : '0' | POS      ;
fragment POS  : [1-9] [0-9]*   ;
```

Lexer rules tagged with **fragment** do not yield a token

- ▶ Can be called only from other lexer rules
- ▶ Enhance grammar readability and maintenance

COMMON LEXICAL STRUCTURES (CONT'D)

Decimal numbers (0, 1, 2, -3, 0.0, 1.4, -2.56, etc.)

- ▶ Match 0 or any sequence of digits starting with 1, potentially prefixed by -
- ▶ Match 0 or any sequence of digits starting with 1, both potentially suffixed by . and any sequence of digits

```
FLOAT           : INT | (INT | '-' '0') '.' DIGIT+ ;
fragment INT    : NAT | '-' POS                      ;
fragment NAT    : '0' | POS                          ;
fragment POS    : POSDIGIT DIGIT*                    ;
fragment DIGIT  : '0' | POSDIGIT                     ;
fragment POSDIGIT : [1-9]                           ;
```

COMMON LEXICAL STRUCTURES (CONT'D)

String literals (strings, sequence of characters, etc.)

- ▶ Match all sequence of characters delimited by double quotes
- ▶ By using Java-like escaping

```
STRING           :  ' " ' STRCHAR* ' " '  ;  
fragment STRCHAR :  ~["\\] | ESC          ;  
fragment ESC     :  '\\ ' [btnfr"'\\" ;
```

Where $\sim \square$ is the complement of the language specified by $\square$

- ▶ Also ANTLR needs escaping!

COMMON LEXICAL STRUCTURES (CONT'D)

Comments (sequences of characters to ignore)

- Match all single and multi-line Java-like comments

```
COMMENT      :  '/' '*' .*? '/' '*' -> skip  ;  
LINE_COMMENT :  '/' '/' ~[\r\n]* -> skip  ;
```

Where `.*?` matches all symbols until `□` is found

- Here `?` indicates a **non-greedy** match (fewest surrounding characters match)

Non-visible characters are also typically skipped

```
WS :  [ \t\r\n]+ -> skip  ;
```

WHITESPACE LANGUAGE

Esoteric¹ language where only spaces, tabs and newlines have meaning

```
Hello, [Space]world! [Space] [Space] [Tab] [Space] [Space] [Tab] [Space] [Space] [Space] [LF]
[Tab] [LF] [Space] [Space]
[Space] [Space] [Space] [Tab] [Tab] [Space] [Space] [Tab] [Space] [Tab] [LF]
[Tab] [LF] [Space] [Space]
[Space] [Space] [Space] [Tab] [Tab] [Space] [Tab] [Tab] [Space] [Space] [LF]
[Tab] [LF] [Space] [Space]
[Space] [Space] [Space] [Tab] [Tab] [Space] [Tab] [Tab] [Space] [Space] [LF]
[Tab] [LF] [Space] [Space]
[Space] [Space] [Space] [Tab] [Space] [Tab] [Tab] [Space] [Space] [LF]
[Tab] [LF] [Space] [Space]
[Space] [Space] [Space] [Tab] [Space] [Space] [Space] [Space] [Space] [LF]
[Tab] [LF] [Space] [Space]
[Space] [Space] [Space] [Tab] [Tab] [Tab] [Space] [Tab] [Tab] [Tab] [LF]
[Tab] [LF] [Space] [Space]
[Space] [Space] [Space] [Tab] [Tab] [Space] [Tab] [Tab] [Tab] [Tab] [LF]
[Tab] [LF] [Space] [Space]
[Space] [Space] [Space] [Tab] [Tab] [Tab] [Space] [Space] [Tab] [Space] [LF]
[Tab] [LF] [Space] [Space]
[Space] [Space] [Space] [Tab] [Tab] [Space] [Tab] [Tab] [Space] [Space] [LF]
[Tab] [LF] [Space] [Space]
[Space] [Space] [Space] [Tab] [Tab] [Space] [Space] [Tab] [Space] [Space] [LF]
[Tab] [LF] [Space] [Space]
[LF] [LF] [LF]
```

Normal devs:



¹https://en.wikipedia.org/wiki/Esoteric_programming_language

DECIMAL EXPRESSION INTERPRETER



EXTEND THE INTEGER EXPRESSION LANGUAGE

Add support for decimal numbers (-1.0, 2.34, ...) and the operators

- ▶ POW denoted by ^, e.g., (-7^3) // *exponentiation*
- ▶ Expressions like $(5.5 + 4) * 2$ and $5.5 + 4 * 2$ are both admitted

✂ Implement an interpreter for the new language in ANTLR

- ▶ Optimize the grammar by applying the advanced patterns from this lecture
- ▶ Define the new language small-step semantics

EXTEND THE INTEGER EXPRESSION LANGUAGE (CONT'D)

Add support for single-line comments

- ▶ Write <!-- to start a comment
- ▶ Such a line is discarded by the lexer, and thus not interpreted

✂ Implement this feature in ANTLR

Q + A



© Michele Pasqua, 2026

This work is licensed under the  **Creative Commons** Attribution-NonCommercial-NoDerivatives 4.0 International license. To view a copy of the license, visit:

<http://creativecommons.org/licenses/by-nc-nd/4.0>

You are free to:

 **Share** – Copy, distribute, and display the work

Under the following terms:

 **Attribution** – You must attribute the work in the manner specified by the author or licensor

 **NonCommercial** – You may not use this work for commercial purposes

 **NoDerivatives** – If you alter, transform, or build upon this work, you may not distribute the modified material

For any reuse or distribution, you must make clear to others the license terms of this work. Any of these conditions can be waived if you get permission from the copyright holder. Your fair use and other rights are in no way affected by the above.